

“초등부 1번. 장애물” 문제 풀이

작성자: 오주원

부분문제 2

장애물 X_1 을 뛰어넘으려면 반드시 $X_1 - 1$ 에서 점프해야 하므로, 우선 시작점 0에서 $X_1 - 1$ 까지 도달하는 최소 이동 횟수를 구한 뒤, 마지막 점프 한 번을 더하면 최소 이동 횟수를 알 수 있다.

시작점에서 $X_1 - 1$ 까지의 거리는 $D = X_1 - 1$ 이다. 이 구간을 이동할 때는 거리 2를 이동하는 점프를 최대한 많이 사용하는 것이 최적이다. D 가 짝수라면 점프만으로 $X_1 - 1$ 까지 갈 수 있고, D 가 홀수라면 한 번은 걸어서 거리 1을 이동해야 한다. 따라서 $X_1 - 1$ 까지의 최소 이동 횟수는 $\lfloor \frac{X_1}{2} \rfloor$ 으로 구해줄 수 있다. 여기에 마지막 점프 한번을 더하면 정답은 $\lfloor \frac{X_1}{2} \rfloor + 1$ 이다.

부분문제 4

장애물을 여러 개 뛰어넘어야 할 때도 기본 원리는 부분문제 2와 동일하다. 각 장애물 X_i 를 뛰어넘기 위해서는 반드시 위치 $X_i - 1$ 에서 점프해서 $X_i + 1$ 에 도착해야 하므로, 장애물 사이 구간을 나누어 생각할 수 있다. 즉, 이전 장애물을 넘은 직후의 위치 $X_{i-1} + 1$ 에서 다음 장애물 직전 위치 $X_i - 1$ 까지 이동하는 최소 이동 횟수를 구하고, 그 후 장애물을 넘기 위한 점프 한 번을 추가하는 방식으로 문제를 해결할 수 있다.

만약 인접한 두 장애물이 정확히 거리 1 차이($X_i + 1 = X_{i+1}$)라면, 이전 장애물을 넘은 직후 착지해야 할 위치가 곧바로 다음 장애물의 위치가 되므로 다음 장애물을 뛰어넘을 수 없게 된다. 이 경우 모든 장애물을 넘는 것은 불가능하므로 -1 을 출력한다.

“초등부 2번 / 중등부 1번. 거울” 문제 풀이

작성자: 반딧불

부분문제 1

$N = 1$ 인 경우, 답은 $2A_1 - s$ 가 된다.

$N = 2$ 인 경우, 거울을 사용하는 순서는 총 두 가지가 있다. 1번 거울을 먼저 사용하는 경우 최종 위치는 $2A_2 - (2A_1 - s)$ 이고, 2번 거울을 먼저 사용하는 경우 최종 위치는 $2A_1 - (2A_2 - s)$ 이다. 둘 중 더 큰 것을 출력하면 된다.

부분문제 2

부분문제 1의 상황에서 2번 거울을 이용하는 것이 더 최적일 조건을 분석하면 $2A_1 - 2A_2 + s \geq 2A_2 - 2A_1 + s$, 즉 $A_1 \leq A_2$ 이다. 따라서 이용하는 거울이 두 개뿐일 때는 더 오른쪽에 있는 거울을 나중에 사용하는 것이 이득이다.

또한, 두 거울을 이용한 뒤의 위치 공식의 형태에서부터, 초기 위치가 오른쪽에 있을 수록 두 개의 거울을 이용한 뒤의 최종 위치도 오른쪽에 있다는 사실을 알 수 있다.

부분문제 2의 상황에서는 이용할 수 있는 거울의 위치가 단 두 가지뿐이다. 왼쪽 거울의 위치 A_1 을 L 이라고 하고, 오른쪽 거울의 위치 A_N 을 R 이라고 하자. 두 가지 종류의 거울 중 적당히 골라 두 번의 위치로 가장 오른쪽으로 보내기 위해서는 먼저 위치 L 의 거울을, 그 다음으로 위치 R 의 거울을 이용하는 것이 최적이다. (초기 위치가 s 일 때 최종 위치가 $2A_2 - 2A_1 + s$ 이므로, A_1 은 작을 수록, A_2 는 클 수록 최적이다.)

따라서 거울을 이 두 종류만 이용할 수 있을 때, 홀수 번째에는 왼쪽 거울을, 짝수 번째에는 오른쪽 거울을 이용하는 것이 최적이다. 마침 두 거울이 정확히 $N/2$ 종류씩 존재하므로, 이 경우가 답이 된다. 이 과정을 직접 시뮬레이션해 답을 $O(N)$ 에 구할 수 있다.

부분문제 3

N 이 짝수이다. N 개 거울의 위치를 이용하는 순서대로 $x_1, x_2, \dots, x_N$ 이라고 하자. 이때 초기 위치 s 에 대해 최종 위치를 식으로 쓰면 다음과 같다.

- 1번 거울까지 이용했을 때: $2x_1 - s$
- 2번 거울까지 이용했을 때: $2x_2 - (2x_1 - s) = 2x_2 - 2x_1 + s$
- 3번 거울까지 이용했을 때: $2x_3 - (2x_2 - 2x_1 + s) = 2(x_1 + x_3) - 2x_2 - s$
- 4번 거울까지 이용했을 때: $2x_4 - (2(x_1 + x_3) - 2x_2 + s) = 2(x_2 + x_4) - 2(x_1 + x_3) + s$
- ...
- N 번 거울까지 이용했을 때: $2(x_2 + x_4 + \dots + x_N) - 2(x_1 + x_3 + \dots + x_{N-1}) + s$ (단, N 은 짝수)

식의 형태를 통해, 홀수 번째에 해당하는 $N/2$ 개의 턴에는 왼쪽에 있는 $N/2$ 개의 거울을 이용하고, 짝수 번째에 해당하는 $N/2$ 개의 턴에는 오른쪽에 있는 $N/2$ 개의 거울을 사용하는 것이 최적임을 알 수 있다. (홀수 번째 턴끼리, 또는 짝수 번째 턴끼리 거울을 바꾸는 것은 최종 결과에 영향이 없음을 알 수 있다.)

거울의 위치가 정렬되어 주어지므로, 답을 $O(N)$ 에 구할 수 있다.

이외에 $A_{N/2} < s < A_{N/2+1}$ 이라는 점을 이용하는 다른 풀이가 있을 수 있다.

부분문제 4

부분문제 3에서 N 이 짝수인 경우의 풀이를 설명하였다. N 이 홀수인 경우, 사용한 거울의 위치 $x_1, x_2, \dots, x_N$ 에 대해 최종 위치는 $2(x_1 + x_3 + \dots + x_N) - 2(x_2 + x_4 + \dots + x_{N-1}) - s$ 가 된다.

식의 형태를 통해 홀수 번째에 해당하는 $(N+1)/2$ 개의 턴에는 오른쪽에 있는 $(N+1)/2$ 개의 거울을, 짝수 번째에 해당하는 $(N-1)/2$ 개의 턴에는 왼쪽에 있는 $(N-1)/2$ 개의 거울을 사용하는 것이 최적임을 알 수 있다.

거울의 위치가 정렬되어 주어지므로, 답을 $O(N)$ 에 구할 수 있다.

“초등부 3번. 통행료” 문제 풀이

작성자: 김기범

부분문제 1

어떤 두 건물 사이를 지나는 가능한 모든 경로를 고려하면, 그러한 것들 중 최소 왕복 비용을 구할 수 있다. 하나의 도로를 두 번 이상 지나면서 최소 왕복 비용일 수는 없으므로, $O(M!)$ 의 시간복잡도로 어떤 건물 쌍에 대한 최소 왕복 비용을 구할 수 있다.

총 M 번 변화하는 도로들의 비용들마다 $O(N^2)$ 번 두 건물 사이의 최소 왕복 비용을 구해야하므로, 전체 시간복잡도 $O(M!N^2M)$ 에 문제를 풀 수 있다.

부분문제 2

임의의 두 건물 사이의 최소 왕복 비용을 구하기 위해 플로이드-워셜 알고리즘을 사용할 수 있다. 해당 알고리즘은 가능한 모든 두 건물 사이의 최소 왕복 비용을 $O(N^3)$ 에 구하도록 해주기 때문에, 도로의 비용이 바뀔 때마다 플로이드-워셜 알고리즘을 사용하면 $O(N^3M)$ 의 시간복잡도로 문제를 해결할 수 있다.

부분문제 3

문제를 거꾸로 생각해서, 처음부터 각 도로들의 통행료가 1인 시점에서부터 도로들을 하나씩 차례대로 통행료를 0으로 설정하는 문제로 생각하자.

맨 처음 시점에 각 건물들 간의 최 왕복 비용을 구하는 것은 플로이드-워셜 알고리즘을 통해 $O(N^3)$ 에 구할 수 있다.

이제, 어떤 도로의 통행료를 0으로 설정하는 경우를 생각하자. 통행료를 0으로 설정하고 나면, 각 건물들 간의 최소 왕복 비용은 변하지 않거나, 혹은 새롭게 통행료가 0으로 변한 도로를 이용하는 경로중 하나가 기존의 최소 왕복 비용보다 효율적일 것이다. 따라서 기존의 최소 왕복 비용을 f , 새로운 최소 왕복 비용을 f' 이라고 하고, 이 도로가 잇는 두 건물을 a, b 라고 할 때, 임의의 두 건물 x, y 에 대해서 다음이 성립한다:

$$f'(x, y) = \min(f(x, y), \min(f(x, a) + f(b, y) + f(x, b) + f(a, y))).$$

따라서, 도로 하나의 통행료가 변할 때마다 최소 왕복 비용을 $O(N^2)$ 에 갱신해줄 수 있으므로 전체 문제를 $O(N^2M)$ 에 해결할 수 있다.

부분문제 4

통행료가 0인 도로만을 통해 오갈 수 있는 건물들 간의 최소 왕복 비용은 0이다. 따라서, 이미 어떤 도로가 잇는 두 건물 사이의 최소 왕복 비용이 0인 경우 해당 도로의 통행료를 0으로 설정하여도 최소 왕복 비용은 바뀌지 않는다.

최소 왕복 비용을 바꾸는 도로의 개수는 $O(N)$ 이기 때문에, 오직 $O(N)$ 개의 도로에 대해서만 최소 왕복 비용을 갱신해주면 되며, 한 번의 갱신은 $O(N^2)$ 에 수행할 수 있기 때문에 전체 문제가 $O(N^3)$ 에 해결된다.

“초등부 4번. 상자 보관” 문제 풀이

작성자: 반딧불

부분문제 1

이 부분문제에서는 $N \leq 6$ 으로 매우 작다. 각 상자에 대해, 해당 상자가 포함하는 다른 상자를 지정(또는 아무 상자도 포함하지 않는다고 지정)하는 모든 경우를 브루트 포스를 통해 모두 시도해볼 수 있다. 구현에 따라 $O(N^N)$ 또는 $O(N!)$ 등의 시간 복잡도가 나올 수 있다.

부분문제 2

이 부분문제에서는 $s_i = c_i + 1$ 로, 모든 상자의 보관 용량이 그 크기보다 1만큼 작다. 따라서 두 상자의 크기가 같으면 두 상자의 보관 용량도 같게 된다.

어떤 상자에도 포함되지 않은 상자 a_1 이 다른 상자 a_2 를 직접 포함하고, 상자 a_2 는 다른 상자 a_3 을 직접 포함하고, ..., 상자 a_{k-1} 는 상자 a_k 를 직접 포함하고, 상자 a_k 는 다른 상자를 포함하지 않는 관계가 존재할 때, 상자 $a_1, a_2, \dots, a_k$ 를 **상자 체인**이라고 정의하자. 자명하게도, 한 상자 체인에 존재하는 상자들은 s_i 값이 모두 달라야 함을 알 수 있다. 따라서, 문제의 답, 즉 모든 상자를 포함하는 데 필요한 최소 상자 체인의 수는 임의의 v 에 대해, $v = s_i$ 인 i 의 개수보다는 커야 한다. 이러한 i 의 개수를 cnt_v 라고 정의하자.

앞에서 보았듯이 답은 최소

$$\max_{1 \leq v} cnt_v$$

이상이어야 한다. 따라서, 만약 정확히 위 개수만큼의 상자 체인으로 모든 상자를 포함할 수 있는 해가 존재한다면, 해당 해가 최적해이다. 부분문제 2의 경우, 이러한 최적해가 존재함을 간단히 알 수 있다.

부분문제 2의 조건에 의해, 상자 k 개가 임의로 주어졌을 때, 모든 상자가 $s_i = c_i + 1$ 을 만족하고 모든 상자의 s_i 값이 다르다면, 이 상자들로 상자 체인을 만드는 것이 항상 가능하다. 만약 s_i 값이 같은 상자들이 존재한다면, s_i 값이 같은 cnt_{s_i} 개의 상자들에 임의의 순서로 1부터 cnt_{s_i} 사이의 라벨을 하나씩 부여하자. 이제 라벨이 같은 상자들끼리 모으면 해당 상자들의 s_i 값은 모두 다르므로, 각각의 묶음으로 상자 체인을 하나씩 만들 수 있다. 라벨의 최댓값은 cnt_v 의 최댓값과 같으므로, 이렇게 만든 해가 최적해이다.

이제 필요한 일은 상자가 하나씩 추가될 때 cnt_v 의 최댓값을 관리하는 것뿐이다. s_i 값이 하나 추가될 때마다 `std::map`과 같은 자료구조에 개수를 늘려서 관리하는 방식으로 해결할 수 있다. 시간 복잡도는 $O(N \log N)$ 이다.

부분문제 3

어떤 상자 a 가 다른 상자 b 를 직접 포함할 때, 쌍 (a, b) 의 **포함 관계**가 있다고 정의하자. 만약 N 개의 상자를 p 개의 상자 체인으로 넣었다면, 총 $N - p$ 개의 포함 관계가 형성되었다는 뜻과 같다. 문제의 조건에 의해, 쌍 (a, b) 의 포함 관계가 형성될 수 있는 조건은 $s_b \leq c_a$ 이다.

문제는 상자 체인의 개수를 최소화할 것을 요구하고 있지만, 이것을 반대로 생각하면, 포함 관계의 수를 최대화해야 하는 문제로도 해석하는 것이 가능하다. 따라서, $s_b \leq c_a$ 조건을 만족하게 최대 몇 개의 포함 관계를 만들 수 있는지 찾으려 한다.

답에 대한 이분 탐색(parametric search)을 이용하자. N 개의 가방이 있을 때, p 개의 포함 관계를 만들 수 있는지 분석하자. 이를 위해 어떤 s_b 값들과 c_a 값들을 고를지 생각해야 한다. 그리디하게 생각하면, 가장

작은 p 개의 s_b 값과 가장 큰 p 개의 c_a 값을 사용하는 것이 최적이다. 저 값들을 쌍을 짓는 최적의 방법은, p 개의 s_b 값들을 오름차순으로 정렬하고, p 개의 c_a 값들도 오름차순으로 정렬한 뒤, 크기 순으로 매칭하는 것이다. 모든 매칭에서 $s_b \leq c_a$ 가 성립한다면 p 개의 포함 관계를 만들 수 있다는 뜻이다.

가방의 개수가 N 개일 때 위 방법을 이용하면 $O(N \log N)$ 시간에 p 의 최댓값을 구할 수 있고, 답을 알아낼 수 있다. 따라서 위 방법을 이용하면 전체 문제를 $O(N^2 \log N)$ 시간에 해결할 수 있다.

부분문제 4

부분문제 3과는 다른 방식으로 접근해 보자. 각 상자 i 를 수직선 위에 (c_i, s_i) 에 해당하는 구간으로 표현할 수 있다. 이때 하나의 상자 체인은 서로 분리된 (쌍마다 교집합이 공집합인) 구간의 집합으로 생각할 수 있다.

다양한 예제를 시도하며 규칙을 찾아 보면 부분문제 2와 비슷하게 문제의 답이 수직선 위의 한 지점이 구간에 포함되는 최대 횟수와 같음을 알 수 있다. 즉, 임의의 v 에 대해, $v \in (c_i, s_i)$ 인 i 의 개수의 최댓값과 같음을 알 수 있다. 일단 이 사실(*)을 참이라고 가정하고, 문제를 해결해 보자.

길이 100의 배열 A 를 관리한다. 어떤 상자 (c_i, s_i) 가 새로 들어올 경우, $A_{c_i}, A_{c_i+1}, \dots, A_{s_i-1}$ 에 1씩 더한다. 해당 연산을 처리한 후, 답은 $\max_i A_i$ 의 값과 같다. 따라서 전체 문제를 $O(N \max s_i)$ 시간에 해결할 수 있다.

이제 풀이의 핵심이 되는 (*)를 증명해 보자. 답이 $\max_i A_i$ 이상이어야 함은 자명하다. 한 위치에서 겹치는 구간이 적어도 $\max_i A_i$ 개 있다는 뜻인데, 이 구간들이 나타내는 상자는 모두 다른 상자 체인에 포함되어야 하기 때문이다. 이제 해당 개수만큼의 상자 체인으로 모든 상자를 포함할 수 있음을 보인다.

다음과 같은 그리디한 접근을 생각한다. 모든 구간을 시작점의 오름차순으로, 시작점이 같다면 끝점의 오름차순으로 정렬한다. 구간을 왼쪽부터 하나씩 보며, 적당한 상자 체인에 넣는 과정을 반복할 것이다. 선택 방법은 다음과 같다.

- 만약 현재 존재하는 상자 체인 중 현재 구간을 넣을 수 있는 것이 있다면, 그러한 상자 체인을 아무거나 선택해 해당 구간을 넣는다.
- 만약 현재 존재하는 상자 체인 중 현재 구간을 넣을 수 있는 것이 없다면 (즉, 현재 구간이 (c_i, s_i) 일 때, 모든 상자 체인에 대해 해당 상자 체인에서 가장 큰 상자의 크기 $s_j > c_i$ 가 성립할 때), 현재 보고 있는 구간만을 포함하는 새로운 상자 체인을 하나 만든다.

위 과정을 통해 상자들을 상자 체인으로 분배할 때 상자 체인의 총 개수가 $\max_i A_i$ 를 넘지 않음을 보인다. 귀류법을 사용하자. 어떤 상자 (c_i, s_i) 를 보는 순간, 새로운 상자 체인을 만들게 되어 상자 체인의 총 개수가 $\max_i A_i + 1$ 개가 되었다고 하자. 이것은 이미 존재하는 $\max_i A_i$ 개의 상자 체인이 모두 현재 상자의 c_i 값보다 s_i 값이 크다는 것을 의미한다. 그러나 상자의 정렬 순서를 고려하였을 때, 이것은 현재 새로 보고 있는 상자의 c_i 값에 대해, $c_i - \epsilon$ 이 포함된 구간의 개수가 $\max_i A_i + 1$ 개임을 의미한다. 이는 A_i 의 정의에 모순이다.

따라서, 위와 같은 그리디한 접근으로 상자 체인이 총 $\max_i A_i$ 개가 되도록 구성할 수 있음을 보였고, 풀이의 정당성이 증명되었다.

부분문제 5

부분문제 4의 풀이는 세그먼트 트리를 이용해 쉽게 최적화할 수 있다. 세그먼트 트리를 이용해 구간 $[c_i, s_i - 1]$ 에 1을 더하고, 전체 최댓값을 찾는 문제로 바꾸는 것이 가능하다. 이렇게 하면 전체 문제를 $O(N \log N)$ 에 해결할 수 있다.